Setting up a new build PC (when the old one is lost or damaged)

This is for SyncLytics staff. It turns a new Windows 11 PC into a full build PC: it can make client releases, issue licence keys, build the desktop till installer and print the client documents. Allow about half a day. Most of it is waiting for downloads.

1. What must already be backed up

You can download everything again except these. Keep **two copies** (for example a USB stick in the office safe and an encrypted cloud folder) and refresh them after every change.

What	Where it lives on the build PC	Why it cannot be replaced
Licence signing key `license-signing-syn-2026-1.json`	`.local-secrets\` in the project folder	Every client's licence was signed with it. A new key cannot renew or replace their licences.
Client pack (documents and the generator)	`client-pack\` in the project folder	Proposals, agreements, invoices and receipts already sent to clients, and the scripts that build them.
Client server passphrases and SSH keys	Password manager	Needed to log in to, update and restore each client's server.
GitHub account (with two-step sign-in recovery codes)	Password manager	The code lives in the private repository `washeeb/Complete-ERP-POS`.

Why: the code is safe on GitHub, but the signing key and client documents are deliberately kept out of it. If they are lost, they are lost.

Check: once a quarter, open the USB copy on another PC and confirm the signing key file and the `client-pack` folder are there and open.

2. Install the tools

Install in this order. Use the default options unless a step says otherwise. Restart the PC at the end.

1. **Git for Windows**: git-scm.com. **Why:** it downloads the code and records changes.
2. **GitHub CLI**: cli.github.com. **Why:** it signs you in to the private repository and opens pull requests.
3. **Node.js 24 LTS**: nodejs.org. **Why:** it builds the till app, the manuals and the client documents.
4. **PHP 8.4 and Composer**: the simplest route is **Laravel Herd** (herd.laravel.com), which installs both. **Why:** the server application, `release:build` and `license:issue` run on PHP.
5. **PostgreSQL 17**: postgresql.org (only needed to run the tests or a local test server). Note the password you choose; you will put it in `server\.env`.
6. **Visual Studio 2022 or later, Community edition**, with the workload **Desktop development with C++**. **Why:** the desktop till is written in Rust, and Rust needs Microsoft's C++ build tools on Windows.
7. **Rust**: rustup.rs, run `rustup-init.exe` and accept the defaults. **Why:** it compiles the desktop till.
8. **Microsoft Word** (for the client documents as PDF). **WebView2** is already part of Windows 11.

Check: open a new PowerShell window and run each of these. Each prints a version number:

```
git --version; gh --version; node --version; php --version; composer --version; cargo --version
```

3. Get the code

```
gh auth login                      # choose GitHub.com, HTTPS, sign in with the browser
cd E:\Businesses\3. Synclytics     # or any folder you prefer
gh repo clone washeeb/Complete-ERP-POS
cd Complete-ERP-POS
```

Why: the repository is private, so you must sign in before you can download it.

4. Put the backups back

1. Copy the `.local-secrets` folder from the USB into the project folder (next to `server` and `pos`).
2. Copy the `client-pack` folder from the USB into the project folder.
3. Tell git to ignore the client pack on this PC (it holds client names and prices):

```
Add-Content .git\info\exclude "client-pack/"
```

Check: `git status` shows nothing new. The `.local-secrets` folder is already ignored by the project.

5. Install the project's own packages

```
cd server; composer install; copy .env.example .env; php artisan key:generate; cd ..
cd pos; npm ci; npx playwright install chromium; cd ..
cd client-pack\generator; npm ci; cd ..\..
```

Open `server\.env` and set `DB_PASSWORD` to your PostgreSQL password.

Why: these download the exact library versions the project was tested with. Playwright's Chromium is what turns the manuals into PDFs.

Check: `cd server; php artisan test` ends with all tests passing (needs the `erp_test` database, see `README.md`).

6. Build the desktop till installer

```
cd pos
npx tauri build
```

The first build takes 10 to 20 minutes (Rust downloads and compiles its libraries; later builds take a few minutes). It also downloads the WiX tool for the `.msi` by itself.

The installers are then in:

- `pos\src-tauri\target\release\bundle\nsis\SES Labs Till_<version>_x64-setup.exe` : give this one to shops.
- `pos\src-tauri\target\release\bundle\msi\SES Labs Till_<version>_x64_en-US.msi` : for IT departments that install by policy.

If `cargo` is "not recognised": close and reopen PowerShell (rustup adds it to the path for new windows), or run `$env:Path = "$env:USERPROFILE\.cargo\bin;$env:Path"` .

Why build it yourself: GitHub also builds the installer (the **Packaging** workflow, artifact `seslabs-till-windows`), but only when its paid minutes are available. The local build does not depend on that.

7. Make a client release and licence

The commands are in `docs/licensing.md` and the cloud case study (Chapter 19 of the installation guide). In short, from the `server` folder:

```
php artisan license:issue --key=..\.local-secrets\license-signing-syn-2026-1.json --customer="<client>" --code=<SYNREQ code> --until=<date> --branches=<n>
php artisan release:build --customer="<client>" --license=<licence id from license:issue> --key=..\.local-secrets\license-signing-syn-2026-1.json --with-pos
```

Check: `license:issue` prints the licence key and its id; a new zip appears in `dist\` . After the client enters the key, their **Licence** page (or `php artisan license:status` on their server) shows the right limits.

8. Manuals and client documents

```
cd pos; npm run manual; npm run training; cd ..
```

```
cd client-pack\generator
node build.js ..
node build-offer.js ..
powershell -File topdf.ps1 -Root ..
```

See `client-pack\generator\README.md` for each document.

9. When the old PC was lost or stolen

1. On github.com → **Settings** → **Sessions** and **SSH and GPG keys**: sign out the old PC and delete its keys. Change the GitHub password.
2. Remove the old PC's SSH key from every client server (`~/.ssh/authorized_keys`) and add the new PC's key.
3. The signing key was on the lost PC. If it may have been copied, make a new one (`php artisan license:keygen --kid=syn-<year>-2 --out=...`), back it up as in section 1, and move every client to it at their next update and renewal (new release, new licence key).

Why: whoever has the old PC may have its saved sign-ins. Removing them closes that door.